

CAS4160 Homework 4: Soft Actor-Critic

[Kim Minsung]

[2020142087]

1 Introduction

DQN is great for discrete action spaces. However, it requires calculating $\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$. Doing this is trivial for discrete action spaces, where you can exhaustively find which of the n actions has the highest Q-value. However, in continuous action spaces, this can be a complex nonlinear optimization problem.

Actor-critic methods get around this by learning an explicit policy, π , that is trained to maximize $\mathbb{E}_{\mathbf{a} \sim \pi(\mathbf{a}|\mathbf{s})} Q(\mathbf{s}, \mathbf{a})$. Throughout this assignment, you will learn to implement one of the most popular algorithms in this line, Soft Actor-Critic (SAC).

All experiments in this assignment can be executed with the following command:

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/<CONFIG>.yaml
```

2 Code Structure Overview

The code structure is similar to that of the previous homework. In this assignment, you mainly work with the following codes:

- `cas4160/scripts/run_hw4.py`: the main training loop for your SAC implementation.
- `cas4160/agents/sac_agent.py`: the SAC learner you will implement.
- `cas4160/networks/state_action_value_critic.py`: a simple MLP-based critic network, $Q(\mathbf{s}, \mathbf{a})$. Note that unlike the DQN's critic, which maps a state to an array of Q-values for all actions, this critic maps $(\mathbf{s}, \mathbf{a})$ to a single Q-value.
- `cas4160/env_configs/sac_config.py`: base configuration (and list of hyperparameters).
- `experiments/sac/*.yaml`: configuration files for the experiments.

3 Training Loop

Before you start implementing SAC, you first need to complete all TODOs in `cas4160/scripts/run_hw4.py`. This is very similar to the DQN script in Homework 3, as both algorithms are off-policy methods!

4 Bootstrapping

As in DQN, we train our critic by “bootstrapping” from a target critic. Using the tuple $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}, d_t)$ (where d_t is the flag for whether the trajectory terminates after this transition), we set a target value y :

$$y \leftarrow r_t + \gamma(1 - d_t)Q_{\phi}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}), \text{ where } \mathbf{a}_{t+1} \sim \pi(\mathbf{a}_{t+1} | \mathbf{s}_{t+1}). \quad (1)$$

Then, we use this bootstrapped target, y , to update our critic, Q_ϕ , by minimizing the L2 loss:

$$\min_{\phi} (Q_{\phi}(s_t, a_t) - y)^2. \quad (2)$$

In practice, we stabilize learning by using a separate *target network*, $Q_{\phi'}$. There are two common strategies for updating the target network, as covered in lectures:

- **Hard update** (implemented in DQN): set $\phi' \leftarrow \phi$ every K steps.
- **Soft update**: ϕ' is continually updated towards ϕ with *Polyak averaging* (exponential moving average). After each update, we perform the following operation (τ is typically very small, e.g., 0.005):

$$\phi' \leftarrow \phi' + \tau(\phi - \phi'). \quad (3)$$

4.1 Implementation

Let's first implement the critic update in `cas4160/agents/sac_agent.py`:

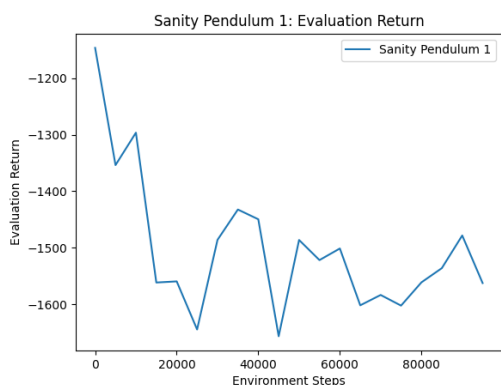
- Implement the bootstrapped critic update explained above in the `update_critic` method. You do not need to implement `next_action_entropy` at this point.
- Update the critic for `num_critic_updates` times in the `update` method.
- Implement soft and hard target network updates in `update`.

4.2 Experiments

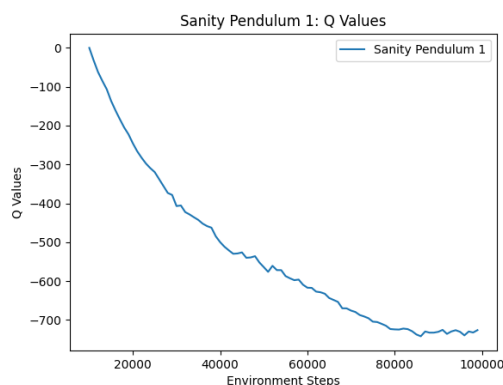
- Train an agent on Pendulum-v1 with the configuration `experiments/sac/sanity_pendulum_1.yaml`.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/sanity_pendulum_1.yaml
```

- It will not get high reward yet as you are not training an actor. But, the Q-values should stabilize at some large negative number (like, -700). If the Q-values go to minus infinity or stay close to zero, there might be something wrong with your code.



(a) Sanity pendulum eval return



(b) Sanity pendulum q values

Figure 1: Sanity Pendulum 1 experiment results with `sanity_pendulum_1.yaml` (critic-only training). (a) Evaluation return over environment steps (remains low since the actor is not trained). (b) Average predicted Q-values over environment steps. The Q-values successfully stabilize at large negative values (around -700).

- The “do-nothing” reward for this environment is about -10 per step. Let’s assume our agent always get “do nothing”, so that they agent receives -10 every step and never terminates. Calculate (approximately) what should be the average Q-value you would expect over training batches considering the discount factor $\gamma = 0.99$.

Answer: Add your explanation.

The do-nothing action in Pendulum-v1 receives a reward of approximately -10 per step. Assuming the agent always selects this action and the episode never terminates, the Q-value of this policy should satisfy the Bellman equation below.

$$Q(s, a) = r + \gamma Q(s', a')$$

Since both the reward r and the next action are constant, this simplifies to:

$$Q = r + \gamma Q$$

Solving for Q :

$$Q(1 - \gamma) = r \implies Q = \frac{r}{1 - \gamma}$$

Substituting $r = -10$ and $\gamma = 0.99$:

$$Q = \frac{-10}{1 - 0.99} = \frac{-10}{0.01} = -1000$$

Therefore, we theoretically expect the average Q-value to converge to approximately -1000. In our experiment (Figure 1(b)), the Q-values stabilize around -700 to -750. This is reasonably close to the theoretical value. The small discrepancy arises because:

- The actual reward in Pendulum-v1 is not strictly constant -10 (it depends on $r = -(\theta^2 + 0.1 \cdot \dot{\theta}^2 + 0.001 \cdot u^2)$).
- The untrained actor still samples some non-zero actions during training, leading to slightly better returns than pure “do-nothing”.
- The soft target network update and initial state distribution also affect the exact convergence point.

5 Entropy Bonus and Soft Actor-Critic

In DQN, we use an ϵ -greedy strategy to decide which action to take at a given time. In continuous spaces, we have several options for generating exploration noise.

One of the most common approaches is providing an *entropy bonus* to encourage the actor to have high entropy (i.e. to be *more random*), scaled by a “**temperature**” coefficient β . For example, in the REPARAMETRIZE case:

$$\mathcal{L}_\pi = Q(s, \mu_\theta(s) + \sigma_\theta(s)\epsilon) + \beta \mathcal{H}(\pi_\theta(\cdot | s)), \text{ where } \epsilon \sim \mathcal{N}(0, 1), \quad (4)$$

where entropy is defined as $\mathcal{H}(\pi(\cdot | s)) = \mathbb{E}_{\mathbf{a} \sim \pi} [-\log \pi(\mathbf{a} | s)]$.

To make sure the entropy term is also factored into the Q-function, we use soft Q target values:

$$y \leftarrow r_t + \gamma(1 - d_t) [Q_\phi(s_{t+1}, \mathbf{a}_{t+1}) + \beta \mathcal{H}(\pi(\cdot | s_{t+1}))]. \quad (5)$$

This soft Q-values result in behaviors where the actor will choose more random actions when it is unsure of what action to take. Please refer to the SAC paper: <https://arxiv.org/abs/1801.01290> for more details.

Note that maximizing entropy $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta]$ requires differentiating *through* the sampling distribution. We can do this via the “reparametrization trick” from lecture – if you’d like a refresher, refer to the Section 7.

5.1 Implementation

In this section, you need to compute (estimate) entropy of a policy and implement soft Q target values in `cas4160/agents/sac_agent.py`:

- Implement the entropy method to calculate the approximate the entropy of an action distribution with one sample:

$$\mathcal{H}(\pi_\theta(\cdot \mid \mathbf{s})) = \mathbb{E}_{\mathbf{a} \sim \pi_\theta} [-\log \pi_\theta(\mathbf{a} \mid \mathbf{s})] \approx -\log \pi_\theta(\hat{\mathbf{a}} \mid \mathbf{s}), \text{ where } \hat{\mathbf{a}} \sim \pi_\theta(\cdot \mid \mathbf{s}). \quad (6)$$

- Add the entropy term to the target critic values in `update_critic`.
- Call `update_actor` in `update` to train the actor to maximize the entropy.

5.2 Experiments

- Train an agent on Pendulum-v1 with the configuration `experiments/sac/sanity_pendulum_2.yaml`, which learns soft Q-values and maximizes a policy entropy:

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/sanity_pendulum_2.yaml
```

- The experiments log entropy during the actor updates. Since the agent only maximizes its entropy, it should achieve (close to) the maximum possible entropy for a 1-dimensional action space. The entropy is maximized when it is a uniform distribution:

$$\mathcal{H}(\mathcal{U}[-1, 1]) = \mathbb{E}[-\log p(x)] = -\log \frac{1}{2} = \log 2 \approx 0.69. \quad (7)$$

Because currently our actor loss **only** consists of the entropy bonus (we have not implemented anything to maximize rewards yet), the entropy should increase until it reaches roughly this level. If the entropy is higher than this, or significantly lower, there can be something wrong with the code.

- There is no result to report in this section.

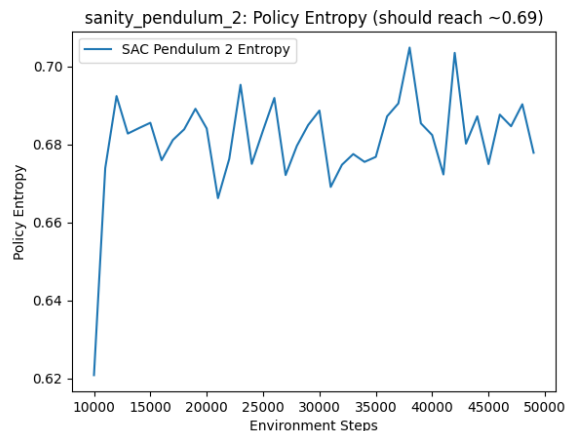


Figure 2: Sanity Pendulum 2 experiment results with `sanity_pendulum_2.yaml`

6 Actor Update with REINFORCE

To train our actor to maximize rewards, we can use the REINFORCE gradient estimator that we used in Homework 2!

The objective of the policy is to maximize the expected return and we need its gradient:

$$\nabla_{\theta} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})} [Q_{\phi}(\mathbf{s}, \mathbf{a})]. \quad (8)$$

As we learned in lecture and Homework 2, we can use REINFORCE to compute the policy gradient:

$$\mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})} [\nabla_{\theta} \log(\pi_{\theta}(\mathbf{a} | \mathbf{s})) Q_{\phi}(\mathbf{s}, \mathbf{a})]. \quad (9)$$

Note that the actions $\mathbf{a}$ are sampled from π_{θ} and we do **not** require real data samples. This means that to reduce the variance of this policy gradient, we can sample more actions from π_{θ} for any given state! You will implement this in your code using the `num_actor_samples` parameter.

6.1 Implementation

In this section, you need to implement the REINFORCE policy gradient estimator in `cas4160/agents/sac_agent.py`:

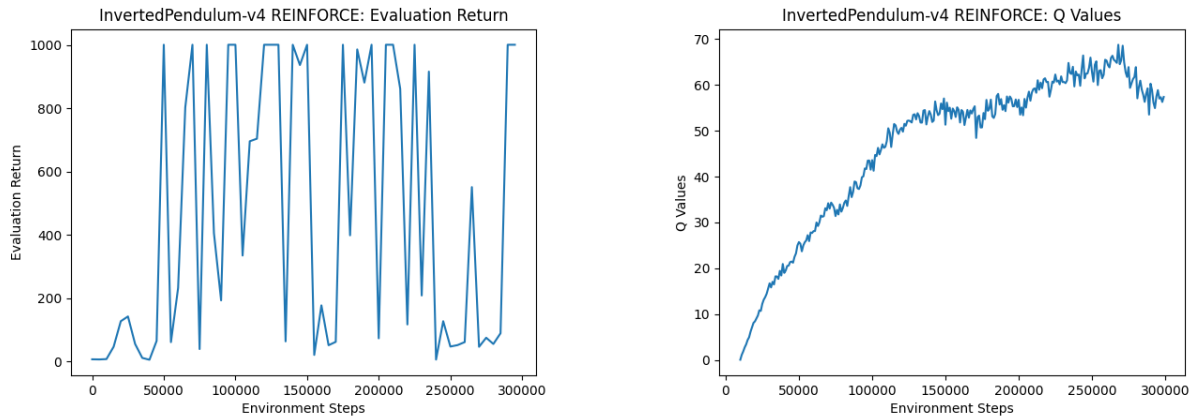
- Implement the REINFORCE gradient estimator in the `actor_loss_reinforce` method.

6.2 Experiments

- Please note that each of the following experiments would take at least an hour.
- Train an agent on `InvertedPendulum-v4` using `sanity_invertedpendulum_reinforce.yaml`. You should achieve a total reward close to 1000, which corresponds to staying upright for all time steps. However, the return may fluctuate, and occasional drops below 1000 are acceptable.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/
sanity_invertedpendulum_reinforce.yaml
```

Answer:



(a) Eval Return(sanity_invertedpendulum_reinforce.yaml) (b) Q values (sanity_invertedpendulum_reinforce.yaml)

Figure 3: InvertedPendulum-v4 results using the REINFORCE policy gradient estimator. (a) Evaluation return over environment steps. The agent successfully learns to keep the pendulum upright, frequently achieving the maximum return of 1000. (b) Average Q-values over environment steps, which steadily increase as the policy improves.

Add your explanation.

In the InvertedPendulum-v4 environment, a return close to 1000 indicates that the pendulum remains balanced (upright) for the entire episode duration. As shown in Figure 3(a), it achieves this maximum return, although there are occasional drops due to the high variance inherent in the REINFORCE gradient estimator.

Figure 3(b) shows that the critic's Q-values increase consistently during training, reflecting the improving quality of the actor's policy. This behavior validates the correctness of the actor_loss_reinforce method, which computes the policy gradient as $\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q_{\phi}(s, a)]$. The fluctuations in evaluation return are expected because REINFORCE is a high-variance estimator. However, the overall upward trend in both return and Q-values demonstrates that the actor is effectively maximizing expected return using only sampled actions from the current policy.

- Train an agent on HalfCheetah-v4 using the provided config (halfcheetah_reinforce1.yaml). Note that this configuration uses only one sampled action for estimating policy gradients. The result should reach positive rewards in 500K steps.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reinforce1.yaml
```

- Train another agent with halfcheetah_reinforce10.yaml. This configuration takes 10 action samples from the actor for computing the REINFORCE gradient. We call this REINFORCE-10 and the single-sample version above REINFORCE-1. REINFORCE-10 should achieve above 500 evaluation returns in 200K steps.
- Plot the results (evaluation return over time) of REINFORCE-1 and REINFORCE-10. Compare and explain your results.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reinforce10.yaml
```

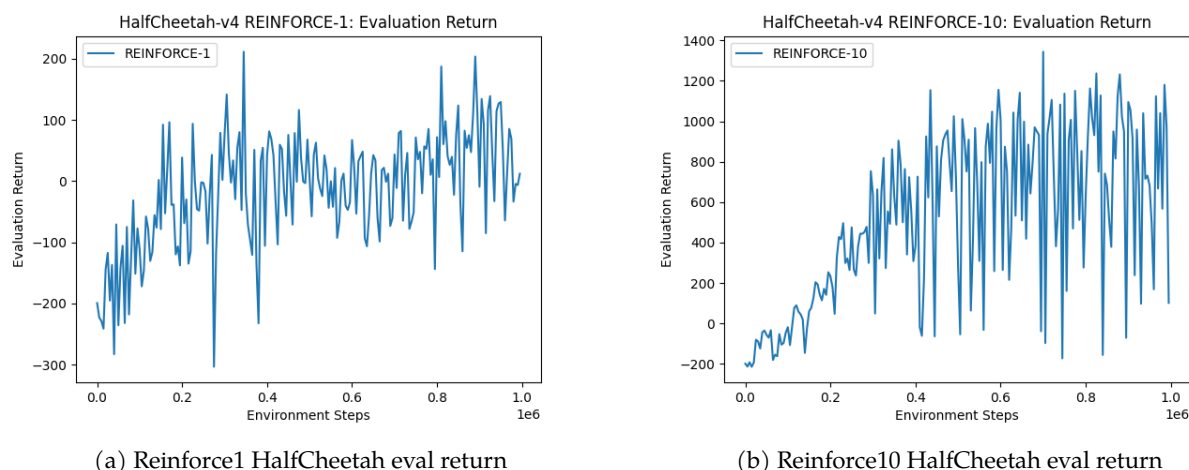


Figure 4: HalfCheetah-v4 evaluation return comparison between REINFORCE-1 and REINFORCE-10. (a) REINFORCE-1 (halfcheetah_reinforce1.yaml, num_actor_samples=1): single-sample policy gradient. (b) REINFORCE-10 (halfcheetah_reinforce10.yaml, num_actor_samples=10): using 10 action samples per state for gradient estimation.

Add your explanation.

REINFORCE is a high-variance policy gradient estimator because it relies on a single Monte-Carlo sample of the action $a \sim \pi_{\theta}(\cdot|s)$. As shown in Figure 4(a), REINFORCE-1 exhibits extremely noisy evaluation returns and learns very slowly. It only reaches positive rewards after approximately 500K steps.

In contrast, REINFORCE-10 (Figure 4(b)) uses 10 independent action samples from the current policy for each state when computing the policy gradient. This significantly reduces the variance of the gradient estimate without introducing bias. As a result, REINFORCE-10 learns much faster and more stably, achieving over 500 evaluation return within 250K~300k steps and reaching returns above 1000 in later stages.

This experiment clearly demonstrates the importance of the num_actor_samples hyperparameter in the REINFORCE implementation. Increasing the number of samples per state is an effective way to mitigate the high variance problem of REINFORCE, leading to substantially better sample efficiency and final performance on complex continuous control tasks such as HalfCheetah-v4.

7 Actor Update with REPARAMETRIZE

REINFORCE works quite well with many samples, but particularly in high-dimensional action spaces, it starts to require a lot of samples to give low variance. We can improve this by using the reparametrized gradient.

If we parameterize π_θ as $\mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon$, where ϵ is normally distributed, then we can write:

$$\nabla_\theta \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})} [Q(\mathbf{s}, \mathbf{a})] = \nabla_\theta \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [Q(\mathbf{s}, \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon)] = \mathbb{E}_{\mathbf{s} \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\nabla_\theta Q(\mathbf{s}, \mu_\theta(\mathbf{s}) + \sigma_\theta(\mathbf{s})\epsilon)] . \quad (10)$$

This gradient estimator often provides a much lower variance, so it can be used with few samples (in practice, just using a single sample works very well).

7.1 Implementation

- Implement `actor_loss_reparametrize()` in `cas4160/agents/sac_agent.py`. Be careful to use the reparametrization trick for sampling (`*.rsample()`)!

7.2 Experiments

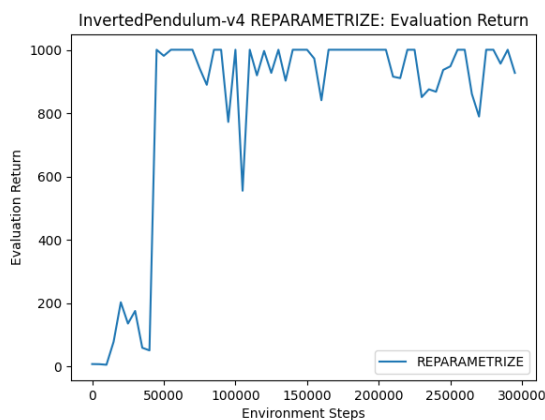
- Please note that each of the following experiments would take at least an hour.
- Make sure you can solve InvertedPendulum-v4 (use `sanity_invertedpendulum_reparametrize.yaml`). You should achieve reward close to 1000, which corresponds to staying upright for all time steps.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/
sanity_invertedpendulum_reparametrize.yaml
```

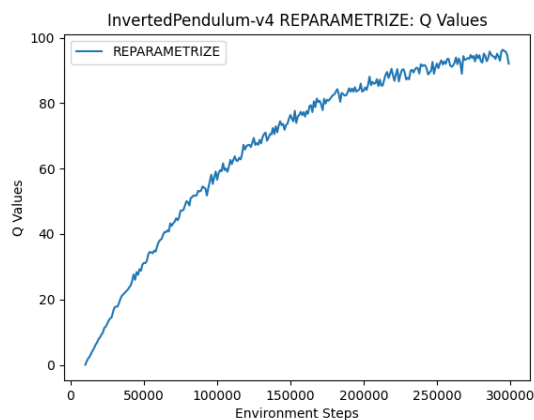
- Train (once again) on HalfCheetah-v4 with `halfcheetah_reparametrize.yaml`. Plot results for **all three gradient estimators** (REINFORCE-1, REINFORCE-10, and REPARAMETRIZE) on the same set of axes, with number of environment steps on the x -axis and evaluation return on the y -axis.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/halfcheetah_reparametrize.
yaml
```

Answer:

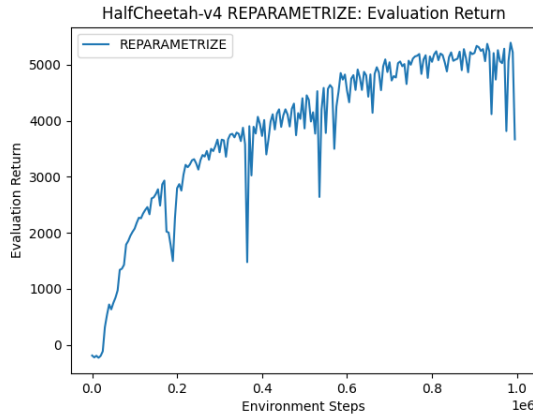


(a) Reparametrize sanity invpendulum eval return

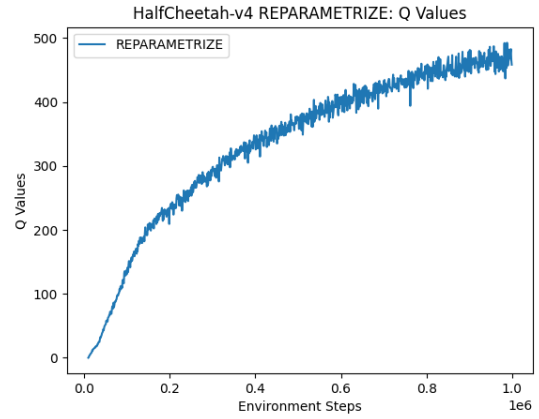


(b) Reparametrize sanity invpendulum q values

Figure 5: InvertedPendulum-v4 results using the reparametrized policy gradient. (a) Evaluation return quickly converges to the maximum value of 1000. (b) Q-values steadily increase as the policy improves.



(a) Reparametrize HalfCheetah eval return



(b) Reparametrize HalfCheetah q values

Figure 6: HalfCheetah-v4 results using the reparametrized policy gradient. (a) Evaluation return. (b) Q-values.

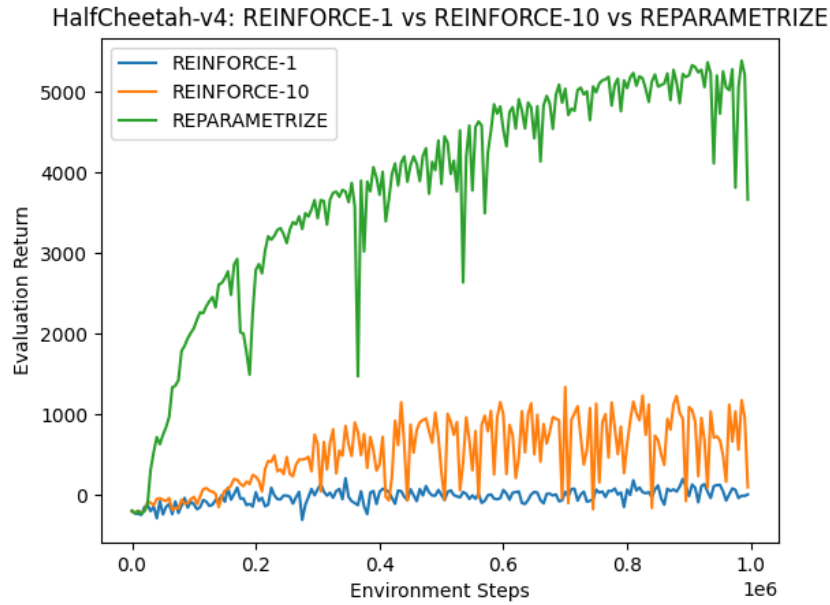


Figure 7: HalfCheetah-v4 evaluation return comparison of three different actor gradient estimators. REINFORCE-1 (num_actor_samples=1), REINFORCE-10 (num_actor_samples=10), and REPARAMETRIZE.

Add your explanation.

The reparametrized gradient estimator significantly outperforms both REINFORCE-1 and REINFORCE-10 variants, as shown in Figure 7.

REINFORCE-1 exhibits high variance and learns very slowly, barely reaching positive returns even after 1M steps. REINFORCE-10 improves performance by using 10 action samples per state to reduce gradient variance, but still shows considerable fluctuations and slower learning compared to REPARAMETRIZE.

In contrast, the reparametrized estimator achieves the highest returns and the most stable learning curve. This is because the reparameterization trick transforms the stochastic policy gradient into a deterministic function of noise $\epsilon \sim \mathcal{N}(0, I)$, allowing the gradient to be computed as $\nabla_{\theta} Q_{\phi}(s, \mu_{\theta}(s) + \sigma_{\theta}(s) \odot \epsilon)$. Unlike REINFORCE, which multiplies a high-variance score function $\nabla_{\theta} \log \pi_{\theta}(a|s)$ with $Q_{\phi}(s, a)$, the reparametrized gradient has much lower variance while remaining unbiased.

This experiment demonstrates that in high-dimensional continuous control tasks like HalfCheetah-v4, the reparameterization trick provides superior sample efficiency and training stability with only a single sample.

- **(Optional)** Train an agent for the Humanoid-v4 environment with `humanoid.yaml` and plot results.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/humanoid.yaml
```

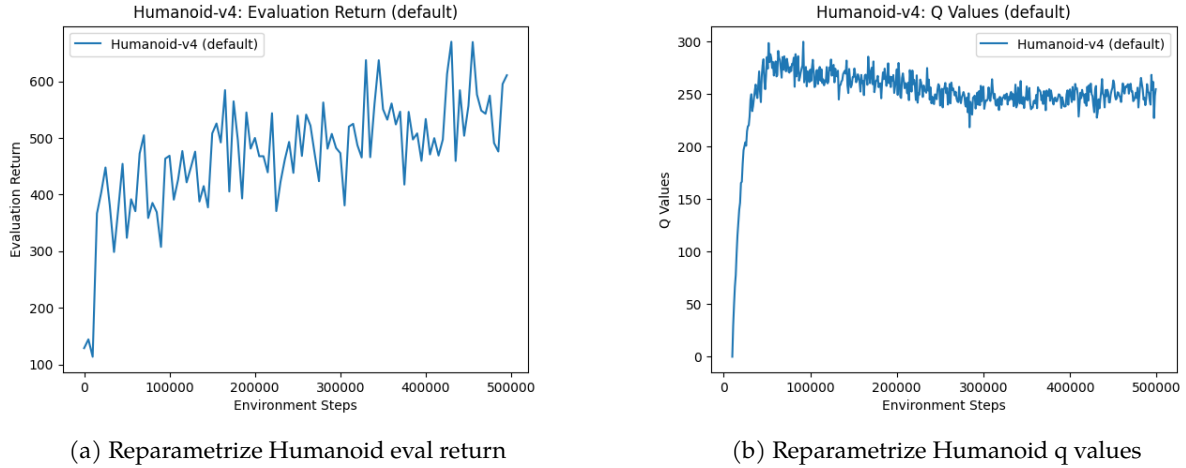


Figure 8: **Replace this with your plot, and add the caption.**

8 Stabilizing Target Values

As in DQN, the target Q with a single critic exhibits *overestimation bias*! There are a few commonly-used strategies to combat this:

- **Double-Q**: learn two critics Q_{ϕ_A}, Q_{ϕ_B} , and keep two target networks $Q_{\phi'_A}, Q_{\phi'_B}$. Then, use $Q_{\phi'_A}$ to compute target values for Q_{ϕ_B} and vice versa (it's slightly different from Double DQN):

$$y_A = r + \gamma Q_{\phi'_B}(s', \mathbf{a}')$$

$$y_B = r + \gamma Q_{\phi'_A}(s', \mathbf{a}')$$

- **Clipped double-Q**: learn two critics Q_{ϕ_A}, Q_{ϕ_B} (and keep two target networks). Then, compute the

target values as $\min(Q_{\phi'_A}, Q_{\phi'_B})$.

$$y_A = y_B = r + \gamma \min(Q_{\phi'_A}(s', a'), Q_{\phi'_B}(s', a'))$$

8.1 Implementation

- Implement double- Q and clipped double- Q in the `q_backup_strategy` function in `sac_agent.py`.

8.2 Experiments

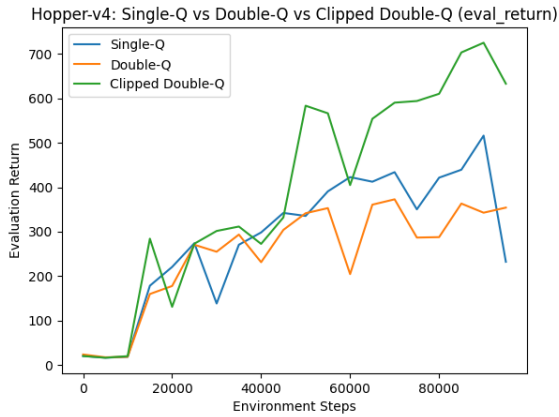
- Please note that each of the following experiments would take around 20 minutes.
- Run single- Q , double- Q , and clipped double- Q on Hopper-v4 using the corresponding configuration files. Which one works best? Plot the logged `eval_return` from each of them as well as `q_values`. Discuss how these results relate to overestimation bias. The default seed should work well, but if not, you may report results using a seed that shows the better tendency.

```
python cas4160/scripts/run_hw4.py -cfg experiments/sac/hopper.yaml --seed 48
python cas4160/scripts/run_hw4.py -cfg experiments/sac/hopper_doubleq.yaml --seed 48
python cas4160/scripts/run_hw4.py -cfg experiments/sac/hopper_clipq.yaml --seed 48
```

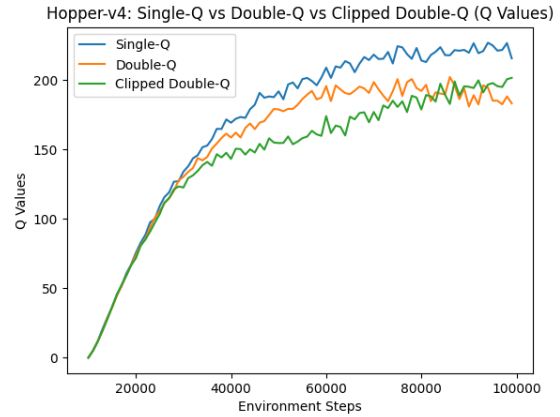
Which one works the best? **Your answer.**

Answer:

Clipped Double- Q works the best.



(a) Hopper eval return comparison



(b) Hopper q values comparison

Figure 9: Hopper-v4 performance comparison among Single- Q , Double- Q , and Clipped Double- Q . (a) Evaluation return over environment steps. (b) Average predicted Q -values over environment steps.

Add your explanation.

Among the three configurations, Clipped Double- Q outperforms both Single- Q and Double- Q , as shown in Figure 9(a). It achieves the highest and most stable evaluation return, reaching approxi-

mately 700 by the end of training. In contrast, Single-Q and Double-Q show lower final performance and more instability.

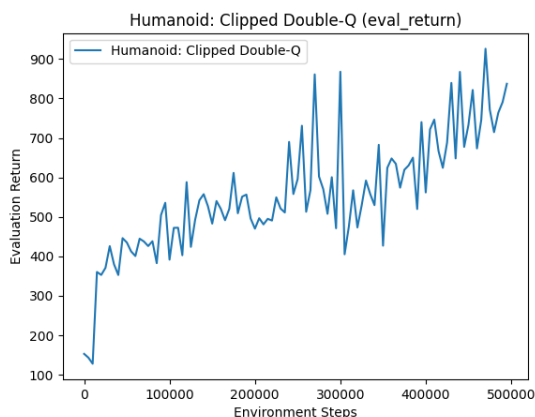
This result is related to overestimation bias. As seen in Figure 9(b).

- Single-Q exhibits the highest Q-values, indicating overestimation of the true action values.
- Double-Q partially mitigates this bias by using two independent critics.
- Clipped Double-Q reduces overestimation most effectively by taking the minimum of the two target critic's predictions.

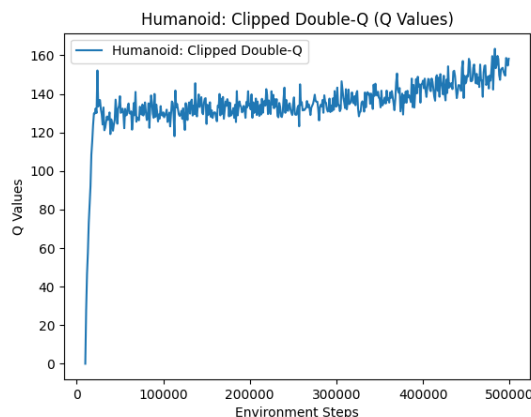
The lower but more accurate Q-values in Clipped Double-Q lead to a more reliable policy update, resulting in superior final performance. This experiment confirms that clipped double-Q is the most effective strategy among the three for solving overestimation bias in off-policy actor-critic methods like SAC.

- **(Optional)** Pick the best configuration among single-Q, double-Q, and clipped double-Q, and run it on Humanoid-v4 using `humanoid.yaml` (**edit the config** to use the best option). Plot the default configuration (in the previous section) and the best configuration together with environment steps on the x -axis and evaluation return on the y -axis.

Note: if you'd like to run training to completion (5M steps, 24h of training), you should get a proper, walking humanoid! You can run with videos enabled by using `-nvid 1`.



(a) Humanoid cipq eval return



(b) Humanoid cipq q values

Figure 10: **Replace this with your plot, and add the caption.**

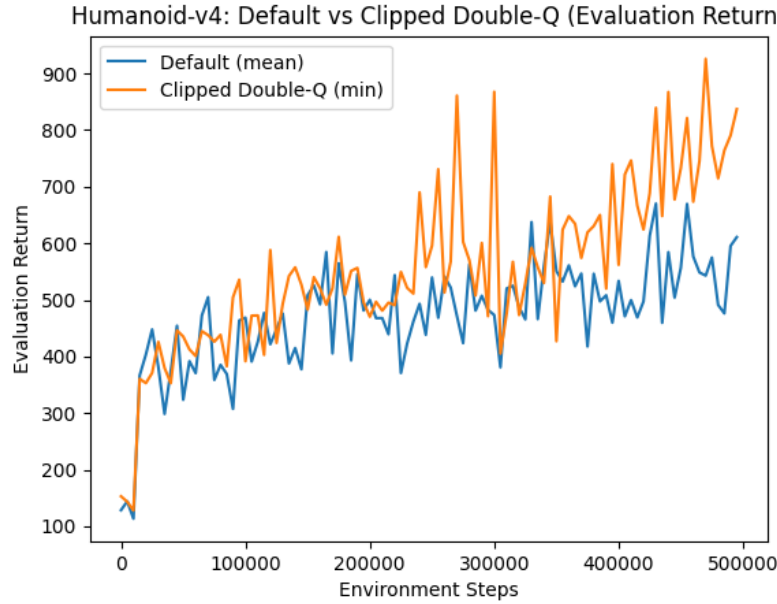


Figure 11: Humanoid-v4 results using the best configuration (Clipped Double-Q). (a) Evaluation return over environment steps. (b) Average predicted Q-values over environment steps.

Clipped Double-Q (`target_critic_backup_type`: min, `num_critic_networks`: 2) was the best configuration based on the Hopper-v4 experiments, where it showed the highest and most stable evaluation return while effectively reducing overestimation bias.

As shown in Figure 10(a), Clipped Double-Q successfully trains a competent policy on the much more challenging Humanoid-v4 environment. The evaluation return steadily increases and reaches approximately 800–900 by 500K steps, demonstrating stable learning without severe collapse. Figure 10(b) shows that the Q-values also increase smoothly and remain bounded, indicating that the clipped target mechanism successfully mitigates the overestimation bias that commonly plagues single-critic or vanilla double-Q methods.

Compared to the default configuration (mean / single critic), Clipped Double-Q produces more reliable Q-value estimates, which in turn leads to better actor updates and higher final performance. This result confirms that the clipped double-Q backup strategy is highly effective even in high-dimensional, complex locomotion tasks like Humanoid-v4.

9 Submission

Please submit the code and the report in a single zip file, `hw4_[YourStudentID].zip`. Do not include videos and logs as the file size should be less than 20MB. The structure of the submission file should be:

```
hw4_[YourStudentID].zip
├── hw4_[YourStudentID].pdf
├── cas4160
│   ├── agents
│   │   └── sac_agent.py
│   └── ...
└── ...
```